
Document Report

Monitoring K8s with Prometheus and Grafana for Cluster and Application Monitoring

Naufal Maulana Hasan

Latar Belakang

Deskripsi

Perkembangan teknologi cloud dan containerisasi telah mendorong banyak perusahaan untuk beralih ke Kubernetes sebagai platform orkestrasi container, berkat fleksibilitas dan kemampuannya dalam mengelola aplikasi dengan skala besar secara otomatis. Kubernetes memungkinkan pengelolaan aplikasi yang efisien dan responsif terhadap perubahan,

Seiring dengan peningkatan adopsi Kubernetes, kebutuhan akan sistem monitoring yang handal juga semakin tinggi. Dalam konteks operasional, perusahaan memerlukan visibilitas penuh terhadap performa dan status cluster Kubernetes untuk memastikan aplikasi berjalan dengan stabil dan sumber daya digunakan secara optimal. Untuk memenuhi kebutuhan ini, kombinasi Prometheus dan Grafana dipilih sebagai solusi monitoring yang andal dan fleksibel.

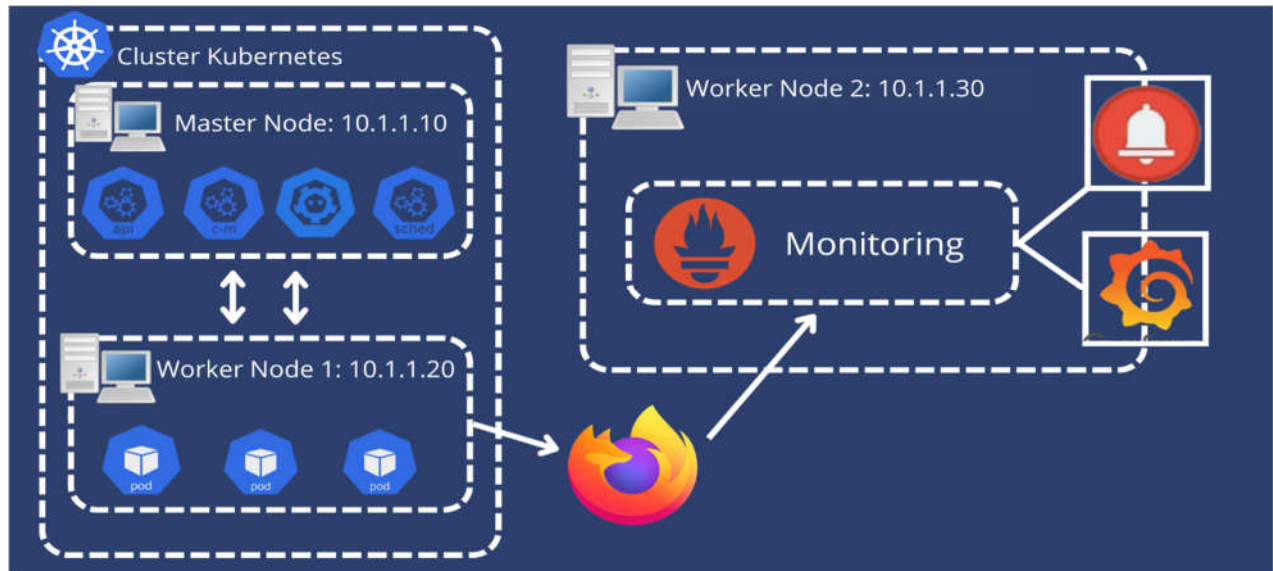
Dimana Prometheus bertindak sebagai sistem pengumpulan data yang mengumpulkan metrik dari berbagai sumber, termasuk node, pod, dan komponen Kubernetes lainnya melalui *exporter*, dan Grafana melengkapi Prometheus dengan menyediakan visualisasi data metrik yang intuitif dan informatif, memungkinkan tim operasional untuk memantau cluster secara keseluruhan sehingga perusahaan dapat memastikan aplikasi yang berjalan dalam cluster Kubernetes tetap andal, aman, dan memenuhi standar performa yang diharapkan.

Tools

- Kubernetes – v1.28.15
- Kubectl
- Kubelet
- Containerd – v1.7.22
- Metrics Server
- Kube State Metrics – v2.13.0
- Node Exporter
- Grafana – v10.2.2
- Prometheus – v2.48.1
- MetalLB – v0.13.7

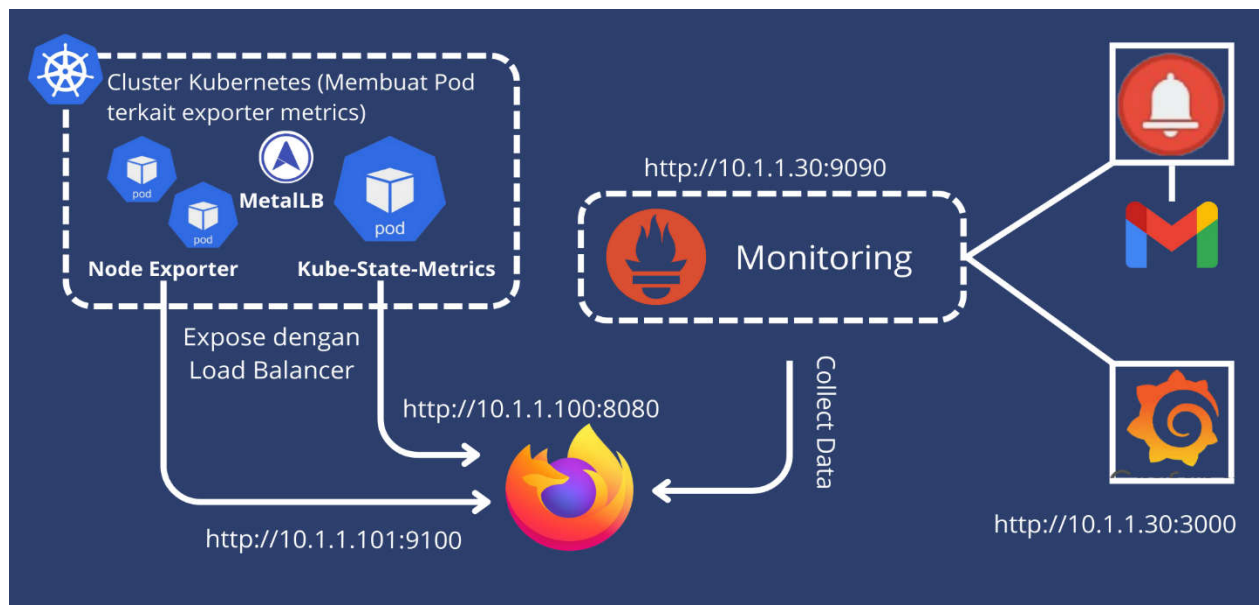
Laporan

Topolog



Terdapat tiga node dimana kedua node dijadikan cluster Kubernetes dan satu node dijadikan sebagai server monitoring, yang nantinya data metrics akan dicollect dari Kubernetes melalui pod

Alur Kerja



Admin membuat daemonset untuk Pod Node Exporter (collect metrics dari node/server fisik mencakup cpu,ram dll), selain itu membuat Pod Kube-State-Metrics (collect metrics dari status dan kondisi objek Kubernetes), dan membuat pod MetalLB setelah tu Pod di expose menggunakan service type LoadBalancer agar bisa diakses oleh Prometheus, nah setelah Prometheus bisa membaca metics, metrics akan di olah dan di visualisasi oleh Grafana.

Lab

Akses Lab : lab16.btech.id		
Hostname	Specification	Keterangan
pod-master	• OS: Ubuntu 22 • CPU: 2 RAM: 2 GB • DISK: 15 GB • IP Node: 10.1.1.10	Master Node (Cluster Kubernetes)
pod-worker1	• OS: Ubuntu 22 • CPU: 2 RAM: 2 GB • DISK: 15 GB • IP Node: 10.1.1.20	Worker Node (Cluster Kubernetes)
pod-worker2	• OS: Ubuntu 22 • CPU: 2 RAM: 2 GB • DISK: 15 GB • IP Node: 10.1.1.30	Monitoring Server

Teori

1. Kubernetes

Kubernetes adalah sistem orkestrasi container open-source yang digunakan untuk mengotomatisasi deployment, scaling, dan pengelolaan aplikasi berbasis container. Kubernetes memungkinkan pengelolaan aplikasi dalam bentuk container di berbagai node secara terdistribusi. Dengan fitur seperti load balancing, auto-scaling, dan rolling updates,

2. Container Runtime (Containerd)

Container runtime adalah komponen di dalam ekosistem Kubernetes yang bertanggung jawab untuk menjalankan dan mengelola container. Containerd adalah salah satu container runtime yang populer, yang menyediakan layanan untuk mengelola siklus hidup container, mulai dari pull image, menjalankan, menghentikan, dan menghapus container. Container runtime ini menjadi dasar bagi Kubernetes dalam menjalankan workload aplikasi.

3. Prometheus

Prometheus adalah sistem monitoring open-source yang dirancang untuk mengumpulkan, menyimpan, dan memproses data metrik dari aplikasi. Dalam konteks Kubernetes, Prometheus mengumpulkan data dari berbagai sumber melalui HTTP endpoints dan menyimpan metrik dalam time-series database. Prometheus juga mendukung alerting, sehingga pengguna dapat menerima notifikasi saat kondisi tertentu tercapai dalam cluster.

4. Grafana

Grafana adalah alat visualisasi data open-source yang digunakan untuk menampilkan data metrik yang dikumpulkan dari Prometheus. Dengan Grafana, pengguna dapat membuat dashboard yang interaktif dan custom untuk memantau kesehatan, kinerja, dan status aplikasi dalam cluster Kubernetes. Grafana memungkinkan pengguna untuk melihat metrik real-time serta menganalisis data historis dari Prometheus.

5. kube-state-metrics

Kube-state-metrics adalah alat yang dikembangkan khusus untuk mengumpulkan data status dari Kubernetes API server, seperti informasi tentang pod, service, deployment, dan lainnya. Kube-state-metrics berfungsi sebagai *exporter* yang menyediakan data dalam format yang dapat dibaca oleh Prometheus.

6. Node Exporter

Node Exporter adalah aplikasi *exporter* yang dirancang untuk mengumpulkan metrik dari sistem operasi, seperti CPU usage, memory usage, disk I/O, dan lainnya. Node Exporter biasanya di-deploy pada setiap node dalam cluster untuk menyediakan informasi tentang kinerja dan kesehatan masing-masing node.

7. Flannel

Flannel adalah salah satu solusi networking yang paling sederhana dan populer untuk Kubernetes. Flannel digunakan sebagai plugin container network interface (CNI) untuk menyediakan jaringan overlay di dalam cluster Kubernetes, yang memungkinkan pod berkomunikasi satu sama lain di seluruh node dalam cluster.

8. ClusterIP, NodePort, dan LoadBalancer Services

Service dalam Kubernetes menyediakan metode untuk mengekspos aplikasi yang berjalan di dalam pod.

- **ClusterIP:** Hanya dapat diakses dari dalam cluster, cocok untuk komunikasi antar pod atau antar komponen internal.
- **NodePort:** Menggunakan port pada node untuk mengekspos layanan ke luar cluster, biasanya untuk keperluan debugging atau akses internal tertentu.
- **LoadBalancer:** Digunakan untuk mengekspos aplikasi ke internet dengan mendefinisikan External Ip dan menjalankan banyak Pod pada satu Ip.

9. DaemonSet

DemonSet adalah resource Kubernetes yang memastikan setiap node dalam cluster menjalankan satu pod dari aplikasi tertentu. Ini ideal untuk aplikasi seperti Node Exporter yang harus berjalan di setiap node agar dapat mengumpulkan metrik dari semua node dalam cluster.

11. MetalLB

MetalLB adalah implementasi LoadBalancer untuk cluster Kubernetes yang berjalan di on-premises (data center pribadi) atau infrastruktur yang tidak menyediakan LoadBalancer bawaan seperti pada cloud providers dengan cara menyediakan alamat IP eksternal

12. Namespace

Namespace adalah cara untuk membagi atau memisahkan sumber daya di dalam Kubernetes menjadi beberapa lingkungan yang terisolasi. Namespace memungkinkan pengaturan aplikasi atau komponen yang berbeda dalam satu cluster Kubernetes tanpa mengganggu satu sama lain.

Langkah Implementasi

Setup VM

1. Ubah Hostname **di semua Node** dengan ost yang unik dan mudah di ingat

```
Node-1 :~$ sudo hostnamectl set-hostname pod-master
Node-2 :~$ sudo hostnamectl set-hostname pod-worker1
Node-3 :~$ sudo hostnamectl set-hostname pod-worker2
```

2. Mapping Hostname ke IP di file */etc/hosts* **untuk semua Node**

```
sudo nano /etc/hosts

127.0.0.1 localhost
10.1.1.10 pod-master
10.1.1.20 pod-worker1
10.1.1.30 pod-worker2
# The following lines are desirable for IPv6 capable hosts
::1 ip6-localhost ip6-loopback
fe00::0 ip6-localnet
ff00::0 ip6-mcastprefix
ff02::1 ip6-allnodes
ff02::2 ip6-allrouters
ff02::3 ip6-allhosts
```

3. Membuat SSH keypair **di semua Nodes** untuk mengkonfigurasi SSH tanpa kata sandi

```
ssh-keygen
```

```
ssh-keygen
```

```
ssh-keygen
```

4. Copy key public dari node master ke semua Node yang

```
ssh-copy-id student@pod-master
ssh-copy-id student@pod-worker1
ssh-copy-id student@pod-worker2
```

Build and Setup Cluster Kubernetes

Note: Execute on all nodes

1. Update dan upgrade package di semua node untuk memastikan semua paket diperbarui

```
sudo apt update
sudo apt upgrade -y
sudo apt autoremove -y
```

2. Instalasi Container Runtime
 - a. Install Dependensi

```
sudo apt install -y curl gnupg2 software-properties-common apt-transport-https ca-certificates
```

- b. Tambahkan Repository Docker dan Instal container runtime (Containerd)

```
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -o /etc/apt/trusted.gpg.d/docker.gpg
sudo add-apt-repository "deb [arch=amd64] https://download.docker.com/linux/ubuntu ${lsb_release -cs} stable"
sudo apt update
sudo apt install -y containerd.io
```

- c. Konfigurasi Agar Containerd Menggunakan Systemd Sebagai Cgroup sehingga membantu Kubernetes dalam mengelola resources secara lebih efektif.

```
containerd config default | sudo tee /etc/containerd/config.toml >/dev/null 2>&1
sudo sed -i 's/SystemdCgroup = false/SystemdCgroup = true/g' /etc/containerd/config.toml
sudo systemctl restart containerd
sudo systemctl enable containerd
```

3. Pengaturan tambahan pada Kernel agar jaringan di antara pod dan container dapat bekerja dengan benar
 - a. Tambahkan Modul Kernel overlay dan br_netfilter

```
cat <<EOF | sudo tee /etc/modules-load.d/k8s.conf
overlay
br_netfilter
EOF

sudo modprobe overlay
sudo modprobe br_netfilter
```

- b. Konfigurasi Parameter Sysctl yang Diperlukan untuk memastikan bahwa firewall dan aturan iptables berlaku pada lalu lintas jaringan antar pod

```
cat <<EOF | sudo tee /etc/sysctl.d/k8s.conf
net.bridge.bridge-nf-call-iptables = 1
net.bridge.bridge-nf-call-ip6tables = 1
net.ipv4.ip_forward = 1
EOF

sudo sysctl --system
```


4. Tambahkan Repository Kubernetes

```
sudo apt-get install -y apt-transport-https ca-certificates
curl -fsSL https://pkgs.k8s.io/core:/stable:/v1.28/deb/Release.key | sudo gpg --dearmor -o /etc/apt/keyrings/kubernetes-apt-keyring.gpg
echo 'deb [signed-by=/etc/apt/keyrings/kubernetes-apt-keyring.gpg] https://pkgs.k8s.io/core:/stable:/v1.28/deb/ /' | sudo tee /etc/apt/sources.list.d/kubernetes.list
```

5. Install kubectl, kubelet & kubeadm

```
sudo apt-get update && sudo apt-get install -y kubelet kubeadm kubectl
sudo apt-mark hold kubelet kubeadm kubectl
```

Note: Execute on master node

6. Nonaktifkan Swap Sementara

```
sudo swapoff -a
```

7. Inisialisasi Cluster dengan Kubeadm dan dengan CIDR jaringan pod Flannel.

```
sudo kubeadm init --pod-network-cidr=10.244.0.0/16
```

8. Salin Konfigurasi Kubectl

```
mkdir -p $HOME/.kube
sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
sudo chown $(id -u):$(id -g) $HOME/.kube/config
```

9. Menginstal plugin Pod jaringan Flannel untuk memungkinkan komunikasi antar pod

```
wget https://raw.githubusercontent.com/coreos/flannel/master/Documentation/kube-flannel.yml
kubectl apply -f kube-flannel.yml
```

10. Tampilkan token dan token-ca-cert-hash untuk join node worker to master

```
sudo kubeadm token list
sudo openssl x509 -pubkey -in /etc/kubernetes/pki/ca.crt | openssl rsa -pubin -outform der 2>/dev/null | openssl dgst -sha256 -hex | sed 's/^.* //'
```

Execute on: worker nodes

11. Join Node Worker to Master

```
sudo swapoff -a
sudo kubeadm join --token [TOKEN] [NODE-MASTER]:6443 --discovery-token-ca-cert-hash sha256:[TOKEN-CA-CERT-HASH]
```

Collecting Metrics (Kube-State-Metrics, Node Exporter, Metrics Server)

1. Metrics Server (Optional, apabila ingin menggunakan fitur HPA)
 - a. Download file *metrics-server* dari repositori resmi

```
wget https://github.com/kubernetes-sigs/metrics-server/releases/latest/download/metrics-server.yaml
```

- b. Tambahkan parameter `--kubelet-insecure-tls` agar metrik dapat berkomunikasi dengan kubelet menggunakan koneksi non-TLS

```
sudo nano ~/metrics-server.yaml

apiVersion: apps/v1
kind: Deployment
metadata:
  labels:
    k8s-app: metrics-server
  name: metrics-server
  namespace: kube-system
spec:
  ...output omitted...
  selector:
    matchLabels:
      k8s-app: metrics-server
  template:
    metadata:
      labels:
        k8s-app: metrics-server
    spec:
      containers:
        - args:
            - --cert-dir=/tmp
            - --secure-port=4443
            - --kubelet-preferred-address-types=InternalIP,ExternalIP,Hostname
            - --kubelet-use-node-status-port
            - --metric-resolution=15s
            - --kubelet-insecure-tls
          image: registry.k8s.io/metrics-server/metrics-server:v0.6.4
        ...output omitted...
```

```
kubectl apply -f metrics-server.yaml
```

2. Buat Namespace Monitoring

```
kubectl create namespace monitoring
```

3. Kube State Metrics

- a. Ambil folder Kube State Metrics dari repository GitHub

```
git clone https://github.com/kubernetes/kube-state-metrics.git
```

- b. Masuk ke directory */examples/standard*

```
cd kube-state-metrics/examples/standard
```

- c. Ubah isi file *kustomization.yaml* pada bagian namespace untuk menggunakan monitoring

```
sudo nano kustomization.yaml

apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
namespace: monitoring

resources:
- cluster-role.yaml
- cluster-role-binding.yaml
- service-account.yaml
- service.yaml
- deployment.yaml
```

- d. Ubah isi file *service.yaml* pada bagian type untuk menggunakan LoadBalancer

```
sudo nano service.yaml

apiVersion: v1
kind: Service
metadata:
  labels:
    app.kubernetes.io/component: exporter
    app.kubernetes.io/name: kube-state-metrics
    app.kubernetes.io/version: 2.13.0
  name: kube-state-metrics
  namespace: monitoring
spec:
  type: LoadBalancer #ubah menjadi LoadBalancer
  ports:
    - name: http-metrics
      port: 80
      targetPort: telemetry
      protocol: TCP
  selector:
    app.kubernetes.io/name: kube-state-metrics
```

4. Node Exporter

a. Buat file node-exporter-Daemonset.yaml

```
sudo nano node-exporter-daemonset.yaml

apiVersion: apps/v1
kind: DaemonSet
metadata:
  name: node-exporter
  namespace: monitoring
  labels:
    app: node-exporter
spec:
  selector:
    matchLabels:
      app: node-exporter
  template:
    metadata:
      labels:
        app: node-exporter
    spec:
      containers:
        - name: node-exporter
          image: prom/node-exporter:latest
          ports:
            - containerPort: 9100
              name: metrics
          resources:
            limits:
              memory: 200Mi
        - name: sys
          mountPath: /host/sys
          readOnly: true
        - name: rootfs
          mountPath: /rootfs
          readOnly: true
      hostNetwork: true
      hostPID: true
      tolerations:
        - effect: NoSchedule
          operator: Exists
      volumes:
        - name: proc
          hostPath:
            path: /proc
        - name: sys
          hostPath:
            path: /sys
        - name: rootfs
          hostPath:
            path: /
```

b. Buat File node-exporter-service.yaml

```
sudo nano node-exporter-daemonset.yaml

apiVersion: v1
kind: Service
metadata:
  name: node-exporter
  namespace: monitoring
  labels:
    app: node-exporter
spec:
  type: LoadBalancer
  selector:
    app: node-exporter
  ports:
    - port: 9100
      targetPort: 9100
```

c. Jalankan Node Exporter

```
kubectl apply -f node-exporter-daemonset.yaml
kubectl apply -f node-exporter-service.yaml
```

5. MetalLB

a. Install MetalLB

```
kubectl apply -f https://raw.githubusercontent.com/metallb/metallb/v0.13.7/config/manifests/metallb-native.yaml
```

b. Buat konfigurasi IP Pool dan L2Advertisement

```
apiVersion: metallb.io/v1beta1
kind: IPAddressPool
metadata:
  name: my-ip-pool
  namespace: metallb-system
spec:
  addresses:
  - 10.1.1.100-10.1.1.110 #sesuaikan dengan network ip yang digunakan untuk node cluster
---
apiVersion: metallb.io/v1beta1
kind: L2Advertisement #membuat IP External dapat diakses oleh pengguna di luar cluster
metadata:
  name: my-l2-advertisement
  namespace: metallb-system
```

Prometheus

1. Cek External IP yang pada Kube State Metrics dan Node Exporter untuk targets scraping

```
pod-master:~$ kubectl get svc -n monitoring
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
kube-state-metrics	LoadBalancer	10.105.32.158	10.1.1.100	8080:30080/TCP,8081:30081/TCP	2d9h
metrics-server	NodePort	10.109.141.154	<none>	443:30443/TCP	5d11h
node-exporter	LoadBalancer	10.98.216.38	10.1.1.101	9100:32273/TCP	34h

2. Pindah ke user root, dan download Prometheus package

```
3. sudo su -
cd /opt
cd prometheus-2.48.1.linux-amd64
```

4. Buat file config file untuk mengatur scrape config (targets exporter yang mau di collect data metricsnya)

```
nano config.yml

global:
  scrape_interval: 10s
  evaluation_interval: 15s

scrape_configs:
  - job_name: 'prometheus-naufal85'
    static_configs:
      - targets: ['10.1.1.30:9090']

  - job_name: 'kube-state-metrics'
    static_configs:
      - targets: ['10.1.1.100:8080']
    metrics_path: /metrics
    scheme: http

  - job_name: 'node_exporter'
    static_configs:
      - targets: ['10.1.1.101:9100']
    metrics_path: /metrics
    scheme: http
```

5. Konfigurasi Prometheus Server sebagai service

```
nano /etc/systemd/system/prometheus_server.service

[Unit]
Description=Prometheus Server

[Service]
User=root
ExecStart=/opt/prometheus-2.48.1.linux-amd64/prometheus \
  --config.file=/opt/prometheus-2.48.1.linux-amd64/config.yml \
  --web.external-url=http://10.1.1.30:9090/ \
  --storage.tsdb.retention.time=15d # menghapus data metrics setiap 15 hari
Restart=on-failure # restart otomatis saat gagal

[Install]
WantedBy=default.target
```

6. Jalankan Prometheus Server service.

```
systemctl daemon-reload
systemctl enable prometheus_server.service
systemctl start prometheus_server.service
systemctl status prometheus_server.service
```

7. Alerting (Opsional)

- a. Pindah ke user root, dan download AlertManager Package

```
sudo su -  
  
cd /opt  
wget https://github.com/prometheus/alertmanager/releases/download/v0.26.0/alertmanager-0.26.0.linux-amd64.tar.gz  
tar xvfz alertmanager-0.26.0.linux-amd64.tar.gz
```

- b. Pindah kedalam directory alert manager

```
cd alertmanager-0.26.0.linux-amd64
```

- c. Konfigurasi AlertManager agar bisa mengirim alert melalui email dengan membuat file config.yaml

```
nano config.yaml  
  
global:  
  resolve_timeout: 5m  
  
route:  
  group_by: ['Alertname']  
  receiver: email-naufal  
  
receivers:  
- name: email-naufal  
  email_configs:  
  - to: "naufalmaulanahasan@gmail.com"  
    from: "naufalmaulanahasan@gmail.com"  
    smarthost: smtp.gmail.com:587  
    auth_username: "naufalmaulanahasan@gmail.com"  
    auth_identity: "naufalmaulanahasan@gmail.com"  
    auth_password: "svcq qyfs obkx cjinm"  
    send_resolved: True
```

- d. Konfigurasi AlertManager sebagai service

```
nano /etc/systemd/system/alert_manager.service
```

```
[Unit]  
Description=Alert Manager  
  
[Service]  
ExecStart=/opt/alertmanager-0.26.0.linux-amd64/alertmanager \\  
  --config.file=/opt/alertmanager-0.26.0.linux-amd64/config.yaml \\  
  --web.external-url=http://10.1.1.30:9093/ \\  
  --log.level=debug  
  
[Install]  
WantedBy=default.target
```


- e. jalankan AlertManager service.

```
./amtool check-config config.yml #cek penulisan didalam file yml
systemctl daemon-reload
systemctl enable alert_manager.service
systemctl start alert_manager.service
systemctl status alert_manager.service
```

- f. pindah ke directory Prometheus

```
cd /opt/prometheus-2.48.1.linux-amd64/
```

- g. Buat rules yang digunakan untuk alerting

```
nano alert_rules.yml

groups:
- name: alert.rules
  rules:

- alert: NodeMemoryPressure
  expr: kube_node_status_condition{condition="MemoryPressure", status="true"} == 1
  for: 5m
  annotations:
    summary: "Node mengalami tekanan memori (Memory Pressure)"
    description: "Node {{ $labels.node }} mengalami tekanan memori selama lebih dari 5 menit."

- alert: NodeNotReady
  expr: kube_node_status_condition{condition="Ready", status="false"} == 1
  for: 5m
  annotations:
    summary: "Node tidak siap (Not Ready)"
    description: "Node {{ $labels.node }} dalam status NotReady selama lebih dari 5 menit."

- alert: PodRestart
  expr: increase(kube_pod_container_status_restarts_total[10m]) > 3
  for: 10m
  annotations:
    summary: "Pod Sering Restart"
    description: "Pod mengalami restart lebih dari 3 kali dalam 10 menit."

- alert: PodShutdown
  expr: kube_pod_status_reason{reason="Shutdown"} > 1
  for: 5m
  annotations:
    summary: "Pod Shutdown"
    description: "Pod {{ $labels.pod }} dalam status mati."
```


- h. Masuk ke file config Prometheus dan tambahkan konfigurasi terkait alertmanager dan nama file rulesnya

```
nano config.yml

alerting:
  alertmanagers:
    - static_configs:
      - targets:
        - 10.1.1.30:9093

rule_files:
  - "alert_rules.yaml"

-----
```

- i. Restart Prometheus Server

```
systemctl restart prometheus_server
```

5. Grafana

1. Pindah ke user root, dan download Grafana package

```
sudo su -

cd /opt
wget https://dl.grafana.com/oss/release/grafana-10.2.2.linux-amd64.tar.gz
ar -zxvf grafana-10.2.2.linux-amd64.tar.gz
```

2. Konfigurasi Grafana sebagai service

```
nano /etc/systemd/system/grafana.service
```

```
[Unit]
Description=Grafana

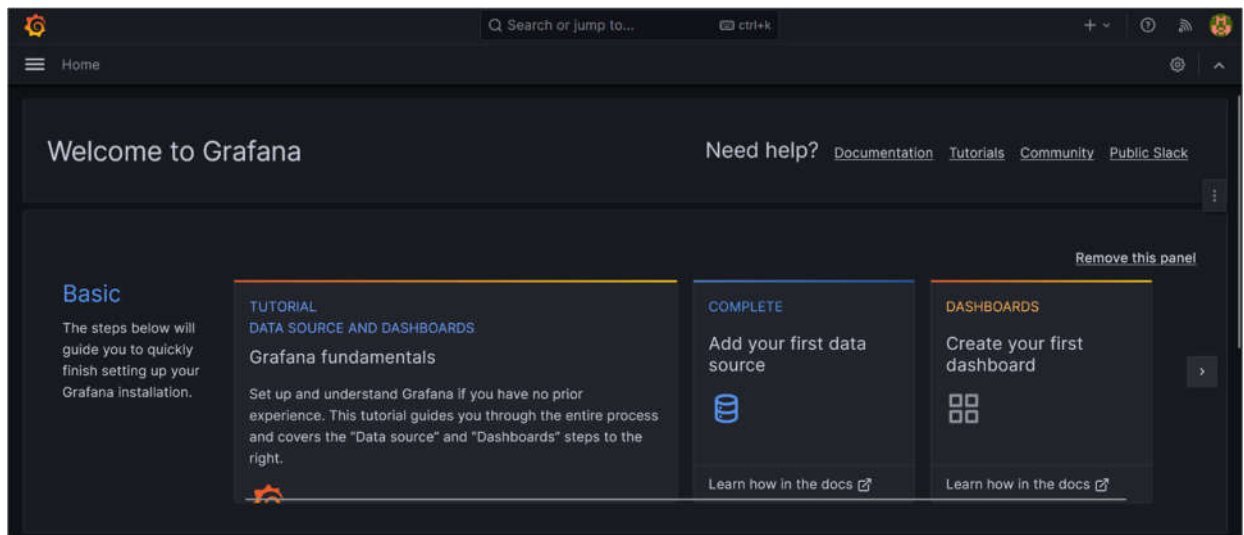
[Service]
User=root
ExecStart=/opt/grafana-v10.2.2/bin/grafana-server -homepath /opt/grafana-v10.2.2/ web

[Install]
WantedBy=default.target
```

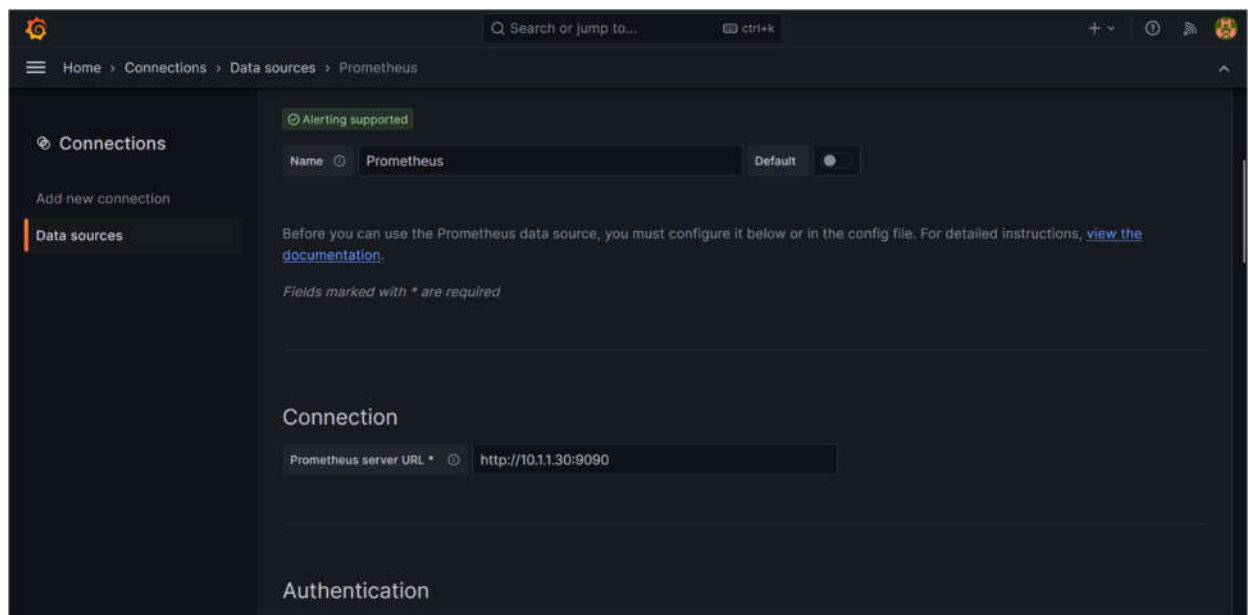
3. Jalankan Grafana Service

```
systemctl daemon-reload
systemctl enable grafana.service
systemctl start grafana.service
systemctl status grafana.service
```

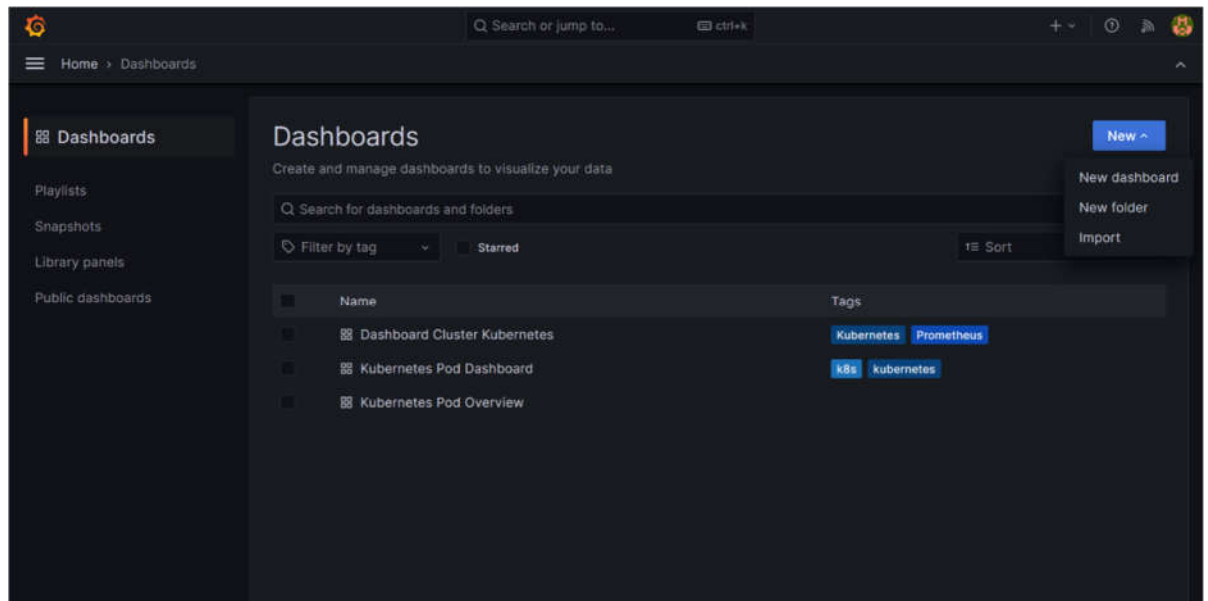
4. Access <http://10.1.1.30:3000/> di browser



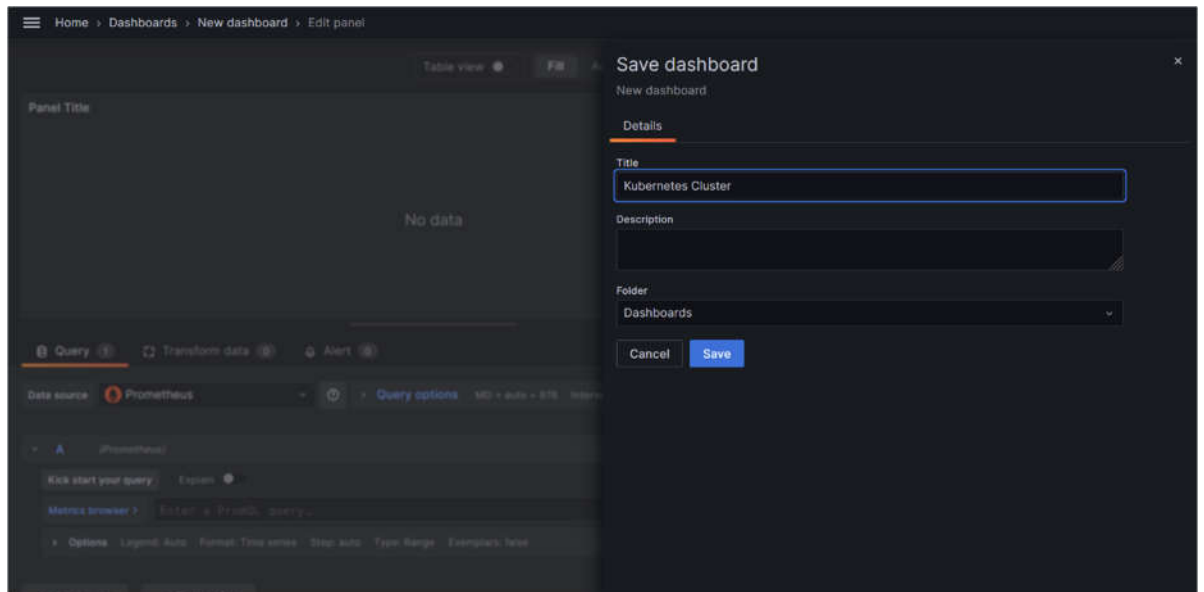
5. Tambahkan prometheus data source. Login to grafana dashboard > Connections > Data sources > Add New data source > Save and Test



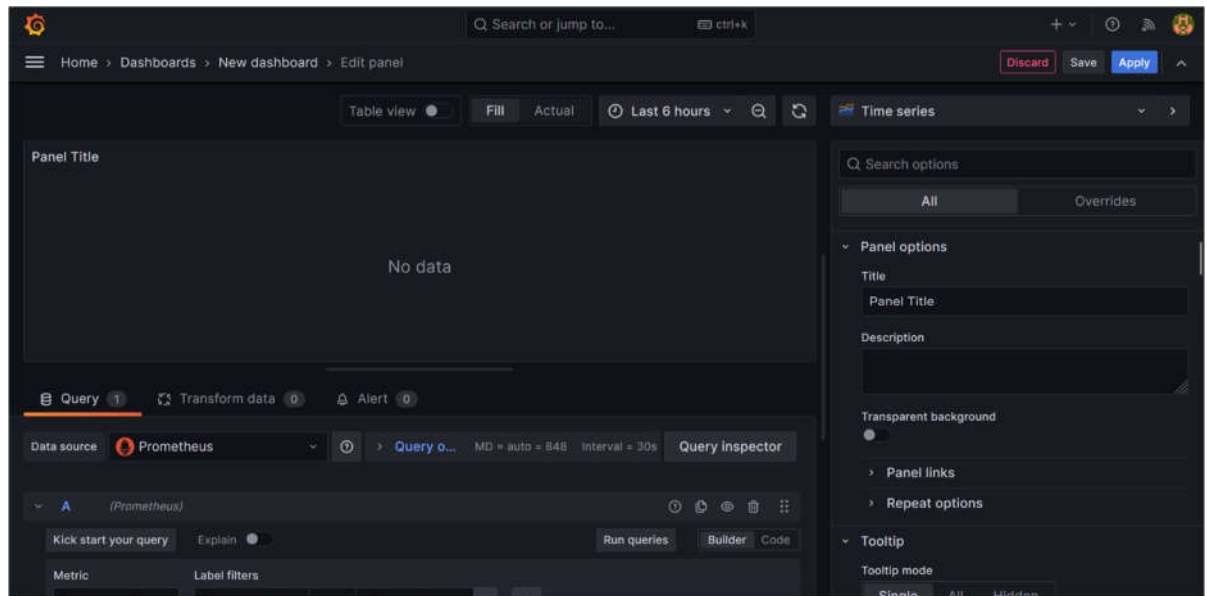
6. Membuat visualisasi dari data metrics (Membuat Dashboard)
 - a. Pergi ke halaman Home > Dashboard > New Dashboard



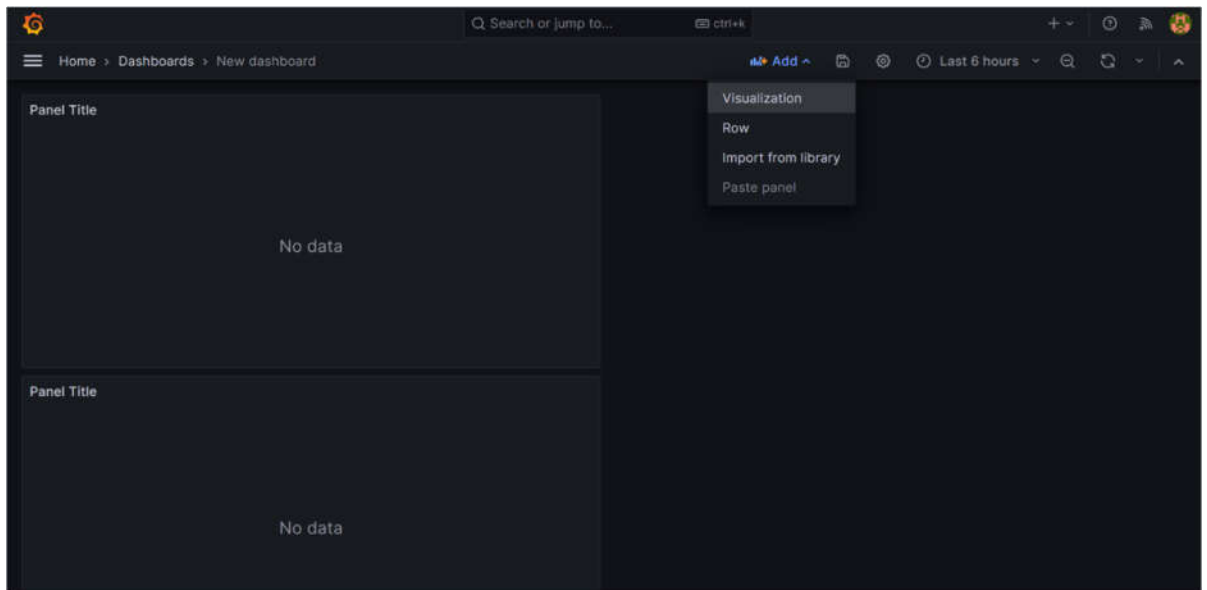
- b. Klik Add Visualization > Prometheus > Masukkan query yang mau di visualisasikan, lalu jika sudah klik save dan berikan title



- c. Klik apply untuk save panel visualization yang telah dibuat

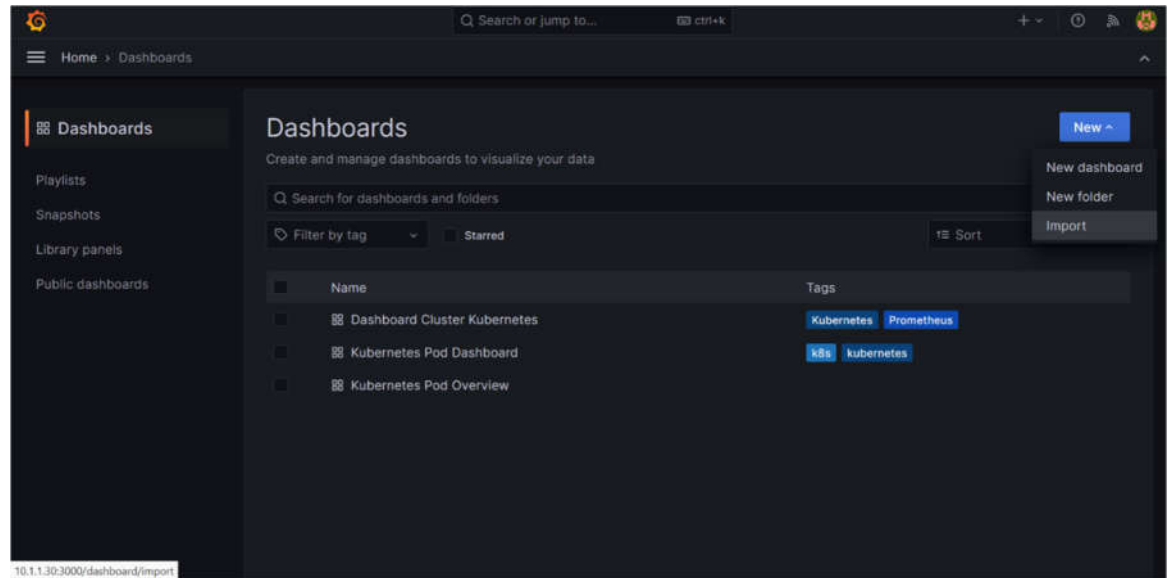


- d. Lakukan berulang dengan cara klik add > visualization dan terakhir jangan lupa save kembali

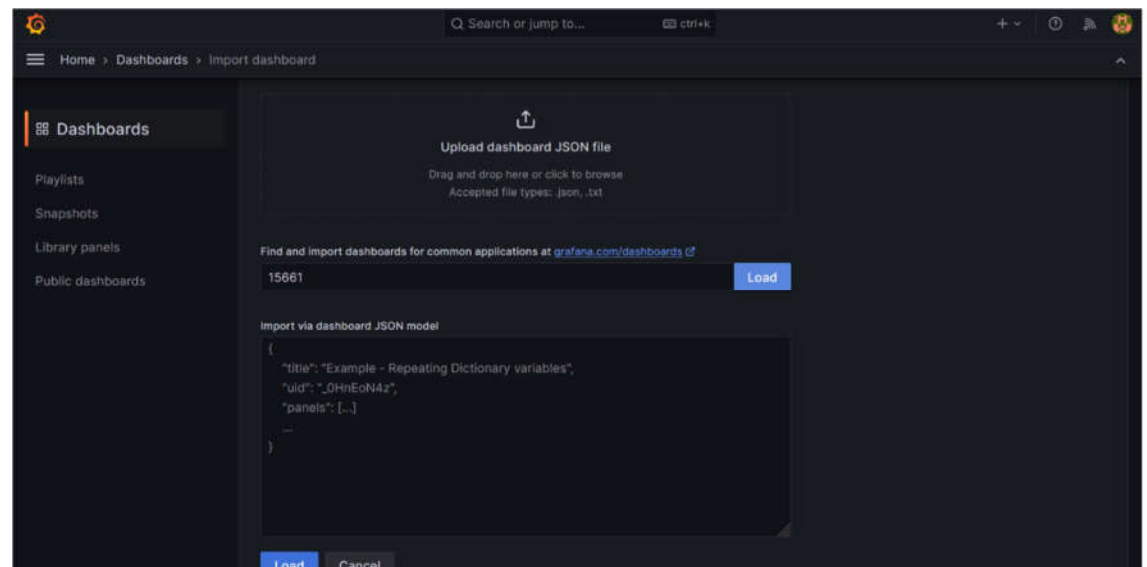


7. Membuat visualisasi dengan mengimport template dashboard dari Grafana Labs (Opsional)

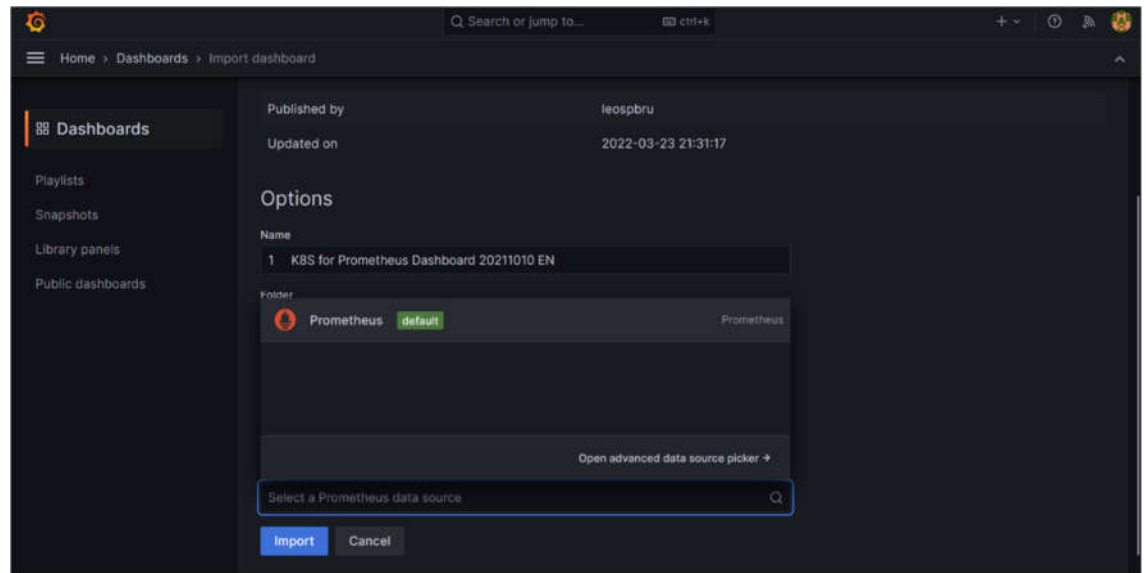
a. Pergi ke halaman Home > Dashboard > Import



b. Masukkan Dashboard URL atau ID dari template Dashboard yang mau di Import
Refrensi template: <https://grafana.com/>



- c. Klik tombol Load, lalu atur nama, Uid, dan Data Sources yang diinginkan



Note: Disarankan menggunakan template Dashboard yang dibuat atau di update pada tahun terbaru, karena kadang query yang diberikan bisa memiliki label dan penamaan yang berbeda, serta disarankan untuk membaca terlebih dahulu apa exporter yang mau di visualisasikan di template.

Langkah Pengujian

Setup VM

1. Uji Koneksi
 - a. Cek Koneksi antar node

```
student@pod-master:~$ ping -c 2 pod-master
PING pod-master (10.1.1.10) 56(84) bytes of data.
64 bytes from pod-master (10.1.1.10): icmp_seq=1 ttl=64 time=0.360 ms
64 bytes from pod-master (10.1.1.10): icmp_seq=2 ttl=64 time=0.083 ms

--- pod-master ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1007ms
rtt min/avg/max/mdev = 0.083/0.221/0.360/0.138 ms

student@pod-master:~$ ping -c 2 pod-worker1
PING pod-worker1 (10.1.1.20) 56(84) bytes of data.
64 bytes from pod-worker1 (10.1.1.20): icmp_seq=1 ttl=64 time=1.53 ms
64 bytes from pod-worker1 (10.1.1.20): icmp_seq=2 ttl=64 time=0.500 ms

--- pod-worker1 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1002ms
rtt min/avg/max/mdev = 0.500/1.017/1.534/0.517 ms

student@pod-master:~$ ping -c 2 pod-worker2
PING pod-worker2 (10.1.1.30) 56(84) bytes of data.
64 bytes from pod-worker2 (10.1.1.30): icmp_seq=1 ttl=64 time=1.74 ms
64 bytes from pod-worker2 (10.1.1.30): icmp_seq=2 ttl=64 time=0.438 ms

--- pod-worker2 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1002ms
rtt min/avg/max/mdev = 0.438/1.087/1.737/0.649 ms
```

- b. Cek Koneksi SSH

```
student@pod-master:~$ ssh student@pod-master "whoami; hostname"
student
pod-master

student@pod-master:~$ ssh student@pod-worker1 "whoami; hostname"
student
pod-worker1

student@pod-master:~$ ssh student@pod-worker2 "whoami; hostname"
student
pod-worker2
```

Kubernetes

1. Bisa membuat Pod dan Resources yang lain

```
student@pod-master:~$ kubectl apply -f nginx-deployment.yaml
deployment.apps/nginx-deployment created
```

2. Memastikan Cluster dalam status ready, dan master maupun worker ditempatkan pada server yang tepat

```
student@pod-master:~$ kubectl get nodes
NAME          STATUS    ROLES    AGE   VERSION
pod-master    Ready     control-plane  8d   v1.28.15
pod-worker1   Ready     <none>      8d   v1.28.15
```

3. Memastikan semua Pod pernting yang berjalan di namespace kube-system dalam status ready

```
student@pod-master:~$ kubectl get pods -n kube-system
NAME                                READY   STATUS    RESTARTS   AGE
coredns-5dd5756b68-jskgq           1/1     Running   0           8d
coredns-5dd5756b68-rv59z           1/1     Running   0           8d
etcd-pod-master                    1/1     Running   0           8d
kube-apiserver-pod-master            1/1     Running   0           8d
kube-controller-manager-pod-master  1/1     Running   0           8d
kube-proxy-7fr7c                   1/1     Running   0           8d
kube-proxy-k86zm                    1/1     Running   0           8d
kube-scheduler-pod-master           1/1     Running   0           8d
```

4. Memeriksa Koneksi API Server: dan memastikan bahwa API server dapat diakses dengan baik

```
student@pod-master:~$ kubectl cluster-info
Kubernetes control plane is running at https://10.1.1.10:6443
CoreDNS is running at https://10.1.1.10:6443/api/v1/namespaces/kube-system/services/kube-dns:dns/proxy
```

5. Kubelet dalam status aktif dan siap digunakan

```
student@pod-master:~$ systemctl status kubelet
● kubelet.service - kubelet: The Kubernetes Node Agent
   Loaded: loaded (/lib/systemd/system/kubelet.service; enabled; vendor preset: enabled)
   Drop-In: /usr/lib/systemd/system/kubelet.service.d
            └─10-kubeadm.conf
   Active: active (running) since Thu 2024-11-07 04:54:13 UTC; 1 week 1 day ago
     Docs: https://kubernetes.io/docs/
  Main PID: 80278 (kubelet)
    Tasks: 14 (limit: 2324)
   Memory: 73.7M
      CPU: 9h 48min 39.997s
   CGroup: /system.slice/kubelet.service
           └─80278 /usr/bin/kubelet --bootstrap-kubeconfig=/etc/kubernetes/bootstrap-kubelet.conf --kubeconfig=/etc/k>
```


Metrics (Kube State Metrics, Node Exporter, MetalLB)

1. Pengujian Kube State Metrics

```
student@pod-master:~$ kubectl get pods -n monitoring -l app.kubernetes.io/name=kube-state-metrics
NAME                                READY   STATUS    RESTARTS   AGE
kube-state-metrics-5464659c6f-88xrw 1/1     Running   0           3d15h

student@pod-master:~$ curl http://10.1.1.100:8080
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>kube-state-metrics</title>
    <style>body {
      font-family: -apple-system,BlinkMacSystemFont,Segoe UI,Roboto,Helvetica Neue,Arial,Noto Sans,Liberation Sans
serif,Apple Color Emoji,Segoe UI Emoji,Segoe UI Symbol,Noto Color Emoji;
      margin: 0;
    }
    header {
      background-color: #e6522c;
      color: #fff;
      font-size: 1rem;
      padding: 1rem;
    }
    main {
      padding: 1rem;
    }
    label {
      display: inline-block;
      width: 0.5em;
    }
  </style>
</head>
<body>
  <header>
    <h1>kube-state-metrics</h1>
  </header>
  <main>
    <h2>Metrics for Kubernetes' state</h2>
    <div>Version: (version=v, branch=, revision=unknown)</div>
    <div>
      <ul>
        <li><a href="/metrics">Metrics</a></li>
        <li><a href="/healthz">Healthz</a></li>
        <li><a href="/livez">Livez</a></li>
      </ul>
    </div>
  </main>
</body>
</html>
```

Note: Menandakan bahwa Kube State Metrics berjalan dengan baik sebagai pod dan mau mengeluarkan data metrics dengan http di IP 10.1.1.100:8080

2. Pengujian Node Exporter

```
student@pod-master:~$ kubectl get pods -n monitoring -l app=node-exporter
NAME                READY   STATUS    RESTARTS   AGE
node-exporter-2glzj  1/1     Running   0           4d2h
node-exporter-5jghb  1/1     Running   0           4d2h

student@pod-master:~$ curl http://10.1.1.101:9100/
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Node Exporter</title>
    <style>body {
      font-family: -apple-system,BlinkMacSystemFont,Segoe UI,Roboto,Helvetica Neue,Arial,Noto Sans,Liberation Sans,sans-
serif,Apple Color Emoji,Segoe UI Emoji,Segoe UI Symbol,Noto Color Emoji;
      margin: 0;
    }
    header {
      background-color: #e6522c;
      color: #fff;
      font-size: 1rem;
      padding: 1rem;
    }
    main {
      padding: 1rem;
    }
    label {
      display: inline-block;
      width: 0.5em;
    }
  </style>
</head>
<body>
  <header>
    <h1>Node Exporter</h1>
  </header>
  <main>
    <h2>Prometheus Node Exporter</h2>
    <div>Version: (version=1.8.2, branch=HEAD, revision=f1e0e8360aa60b6cb5e5cc1560bed348fc2c1895)</div>
    <div>
      <ul>
        <li><a href="/metrics">Metrics</a></li>
      </ul>
    </div>
  </main>
</body>
</html>
```

Note: Menandakan bahwa Node Exporter dijalankan dengan Daemonset dan berjalan dengan baik di setiap node sebagai pod dan dapat mengeluarkan data metrics dengan http di IP 10.1.1.101:9100

3. Pengujian pada Metrics Server

```
student@pod-master:~$ kubectl top nodes
NAME          CPU(cores)   CPU%   MEMORY(bytes)   MEMORY%
pod-master    229m         11%    1426Mi          76%
pod-worker1   74m          3%     1296Mi          69%

student@pod-master:~$ kubectl top pods --all-namespaces
NAMESPACE     NAME                                                    CPU(cores)   MEMORY(bytes)
kube-flannel   kube-flannel-ds-d5xvv                                  15m          11Mi
kube-flannel   kube-flannel-ds-nchq2                                  17m          11Mi
kube-system    coredns-5dd5756b68-jskgq                              3m           13Mi
kube-system    coredns-5dd5756b68-rv59z                              5m           14Mi
kube-system    etcd-pod-master                                         38m          45Mi
kube-system    kube-apiserver-pod-master                             127m         259Mi
kube-system    kube-controller-manager-pod-master                    31m          54Mi
kube-system    kube-proxy-7fr7c                                        2m           19Mi
kube-system    kube-proxy-k86zm                                        1m           13Mi
kube-system    kube-scheduler-pod-master                              7m           24Mi
metallb-system controller-67d9f4b5bc-qbp5d                             2m           14Mi
metallb-system speaker-kks2k                                           9m           16Mi
metallb-system speaker-s66xc                                           6m           16Mi
monitoring     kube-state-metrics-5464659c6f-88xrw                    4m           15Mi
monitoring     metrics-server-596474b58-24tqr                          6m           19Mi
monitoring     node-exporter-2glzj                                     0m           7Mi
monitoring     node-exporter-5jghb                                     7m           9Mi
```

Note: Menandakan bahwa Metrics Server berjalan dengan baik dan bisa mengambil data metrics

4. Pengujian pada MetalLB (Alat untuk LoadBalancer)

```
student@pod-master:~$ kubectl get svc -n monitoring
NAME                TYPE          CLUSTER-IP    EXTERNAL-IP    PORT(S)                                     AGE
kube-state-metrics  LoadBalancer 10.105.32.158  10.1.1.100     8080:30080/TCP,8081:30081/TCP             3d15h
metrics-server      NodePort      10.109.141.154 <none>         443:30443/TCP                             6d17h
node-exporter        LoadBalancer 10.98.216.38   10.1.1.101     9100:32273/TCP                           2d16h

student@pod-master:~$ kubectl get pods -n metallb-system
NAME                READY   STATUS    RESTARTS   AGE
controller-67d9f4b5bc-qbp5d  1/1    Running   1 (2d3h ago)  2d3h
speaker-kks2k          1/1    Running   0           2d3h
speaker-s66xc          1/1    Running   0           2d3h
```

Note: Menandakan MetalLB berjalan dengan baik dan bisa memberikan External IP yang sudah ditentukan yaitu diantara 10.1.1.100 – 10.1.1.110, serta layanan bisa diakses di browser dengan IP External

Prometheus

1. Pengujian pada status system

```
student@pod-worker2:~$ sudo systemctl status prometheus_server
● prometheus_server.service - Prometheus Server
   Loaded: loaded (/etc/systemd/system/prometheus_server.service; enabled; vendor preset: enabled)
   Active: active (running) since Thu 2024-11-14 09:27:40 UTC; 11h ago
     Main PID: 161341 (prometheus)
        Tasks: 10 (limit: 2324)
       Memory: 124.9M
          CPU: 4min 6.992s
      CGroup: /system.slice/prometheus_server.service
              └─161341 /opt/prometheus-2.48.1.linux-amd64/prometheus --config.file=/opt/prometheus-2.48.1.linux-amd64/co>

Nov 14 17:00:06 pod-worker2 prometheus[161341]: ts=2024-11-14T17:00:06.389Z caller=compact.go:464 level=info component=>
Nov 14 17:00:06 pod-worker2 prometheus[161341]: ts=2024-11-14T17:00:06.397Z caller=db.go:1619 level=info component=tsdb>
Nov 14 17:00:06 pod-worker2 prometheus[161341]: ts=2024-11-14T17:00:06.403Z caller=db.go:1619 level=info component=tsdb>
Nov 14 17:00:06 pod-worker2 prometheus[161341]: ts=2024-11-14T17:00:06.416Z caller=db.go:1619 level=info component=tsdb>
Nov 14 19:00:04 pod-worker2 prometheus[161341]: ts=2024-11-14T19:00:04.972Z caller=compact.go:523 level=info component=>
Nov 14 19:00:04 pod-worker2 prometheus[161341]: ts=2024-11-14T19:00:04.987Z caller=head.go:1299 level=info component=ts>
Nov 14 19:00:04 pod-worker2 prometheus[161341]: ts=2024-11-14T19:00:04.990Z caller=checkpoint.go:100 level=info compone>
Nov 14 19:00:05 pod-worker2 prometheus[161341]: ts=2024-11-14T19:00:05.276Z caller=head.go:1267 level=info component=ts>
Nov 14 21:00:05 pod-worker2 prometheus[161341]: ts=2024-11-14T21:00:05.046Z caller=compact.go:523 level=info component=>
Nov 14 21:00:05 pod-worker2 prometheus[161341]: ts=2024-11-14T21:00:05.067Z caller=head.go:1299 level=info component=ts>
lines 1-20/20 (END)
```

2. Pengujian apakah Prometheus dapat diakses di browser untuk dashboardnya dan mengeluarkan targets yang sudah di konfigurasi untuk di collect datanya

 Prometheus Alerts Graph Status Help

Targets

All scrape pools All Unhealthy Collapse All Filter by endpoint or labels Unknown Unhealthy Healthy

kube-state-metrics (1/1 up) show less

Endpoint	State	Labels	Last Scrape	Scrape Duration	Error
http://10.1.1.100:8080/metrics	UP	instance="10.1.1.100:8080" job="kube-state-metrics"	2.210s ago	13.685ms	

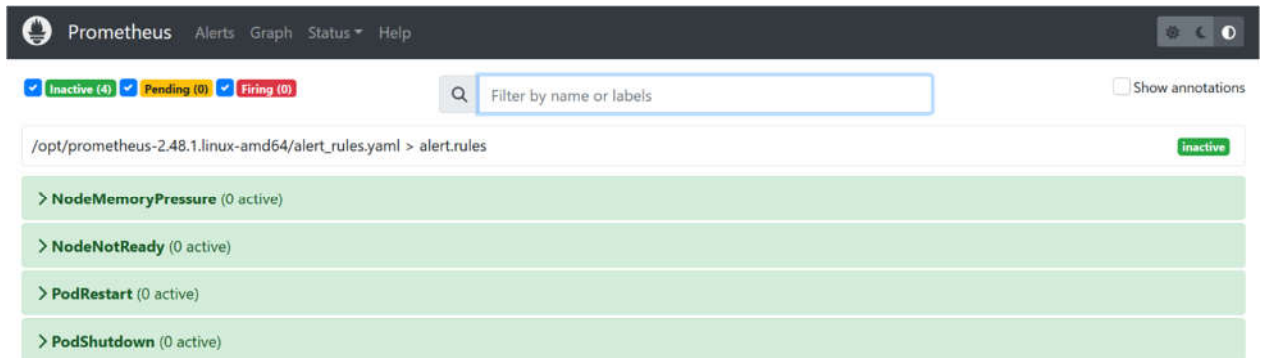
node_exporter (1/1 up) show less

Endpoint	State	Labels	Last Scrape	Scrape Duration	Error
http://10.1.1.101:9100/metrics	UP	instance="10.1.1.101:9100" job="node_exporter"	5.916s ago	242.299ms	

prometheus-naufal85 (1/1 up) show less

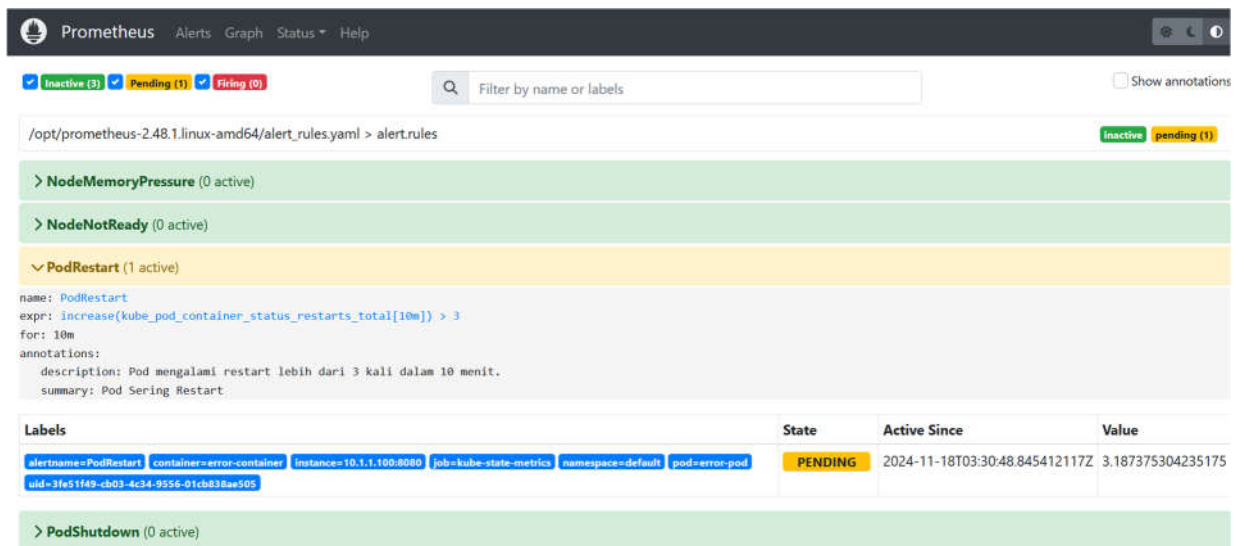
Endpoint	State	Labels	Last Scrape	Scrape Duration	Error
----------	-------	--------	-------------	-----------------	-------

- Pengujian apakah Prometheus menerima Alert yang sudah di konfigurasi dan diatur rulesnya di Alertmanager dan memastikan alert berjalan sesuai query



The Prometheus Alerts page shows a list of alerts under the path `/opt/prometheus-2.48.1.linux-amd64/alert_rules.yaml > alert.rules`. The status bar at the top indicates 4 Inactive, 0 Pending, and 0 Firing alerts. The list contains four alerts, all of which are inactive:

- > NodeMemoryPressure (0 active)
- > NodeNotReady (0 active)
- > PodRestart (0 active)
- > PodShutdown (0 active)



The Prometheus Alerts page now shows one alert in a pending state. The status bar indicates 3 Inactive, 1 Pending, and 0 Firing alerts. The `PodRestart` alert is expanded, showing its configuration and a table of its labels and state.

Alert Configuration:

- name:** PodRestart
- expr:** `increase(kube_pod_container_status_restarts_total[10m]) > 3`
- for:** 10m
- annotations:**
 - description:** Pod mengalami restart lebih dari 3 kali dalam 10 menit.
 - summary:** Pod Sering Restart

Labels	State	Active Since	Value
<code>alertname=PodRestart</code> <code>container=error-container</code> <code>instance=10.1.1.100:8080</code> <code>job=kube-state-metrics</code> <code>namespace=default</code> <code>pod=error-pod</code> <code>uid=3fe51f49-cb03-4c34-9556-01cb838ae505</code>	PENDING	2024-11-18T03:30:48.845412117Z	3.187375304235175

[FIRING:1] (PodRestart error-container 10.1.1.100:8080 kube-state-metrics default error-pod 3fe51f49-cb03-4c34-9556-01cb838ae505) [inbox x](#)

 **naufalmaulanahasan@gmail.com**
to me ▾

10:41AM (0 minutes ago) ☆ 🗨 ↶ ⋮

1 alert for

[View in Alertmanager](#)

[1] Firing

Labels

alertname = PodRestart
container = error-container
instance = [10.1.1.100:8080](#)
job = kube-state-metrics
namespace = default
pod = error-pod
uid = 3fe51f49-cb03-4c34-9556-01cb838ae505

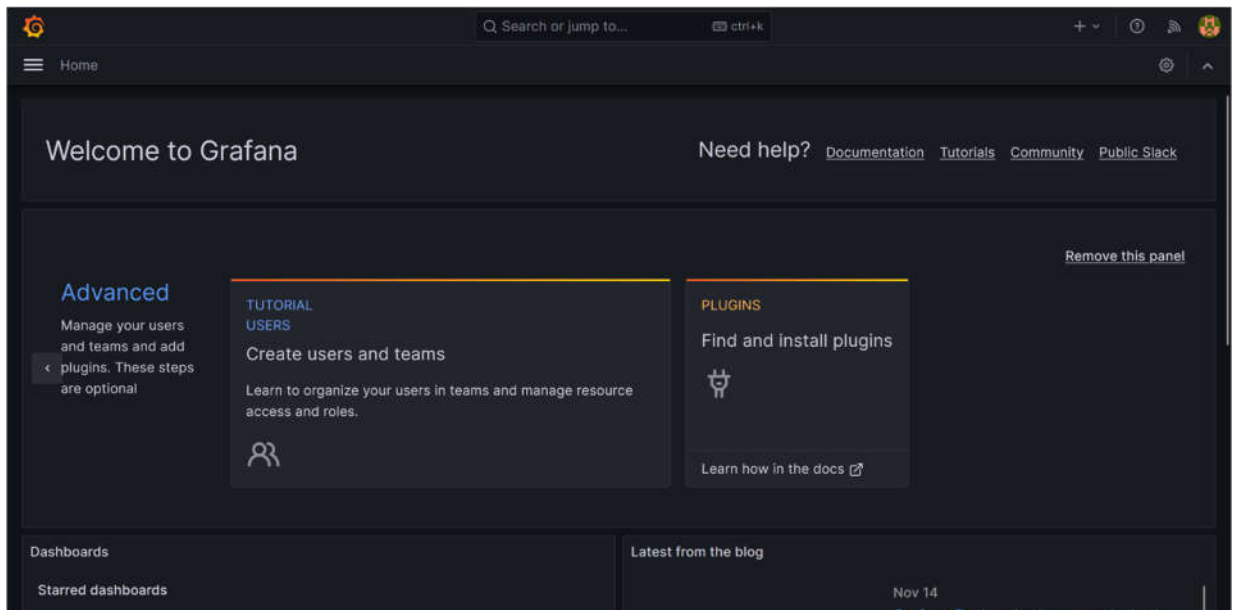
Annotations

description = Pod mengalami restart lebih dari 3 kali dalam 10 menit.
summary = Pod Sering Restart

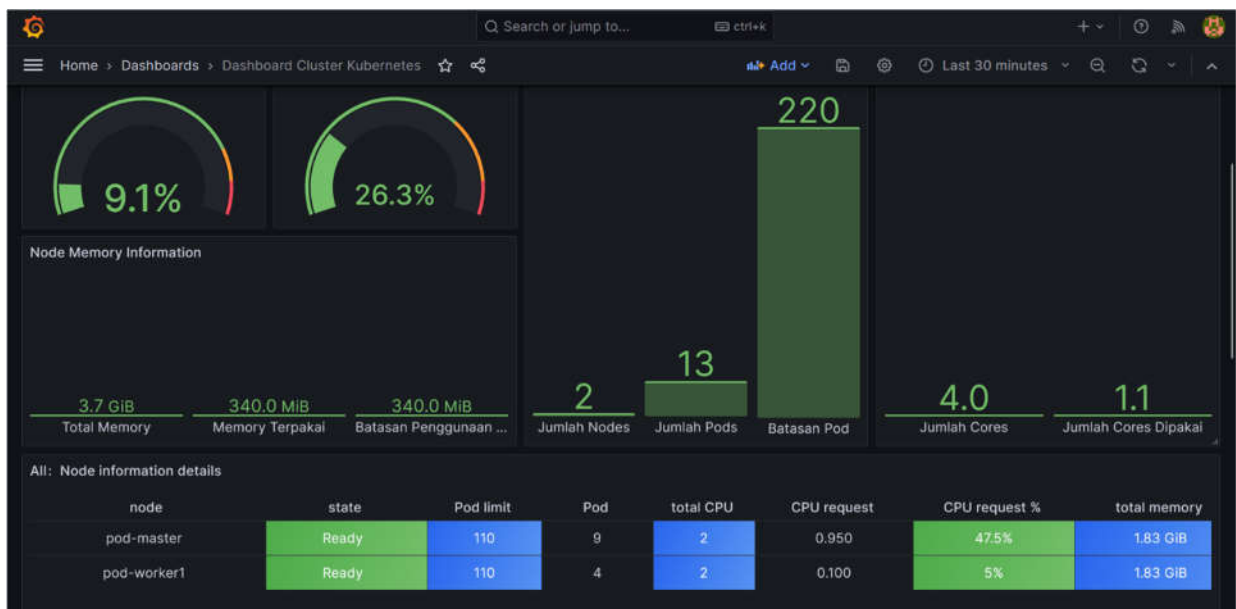
[Source](#)

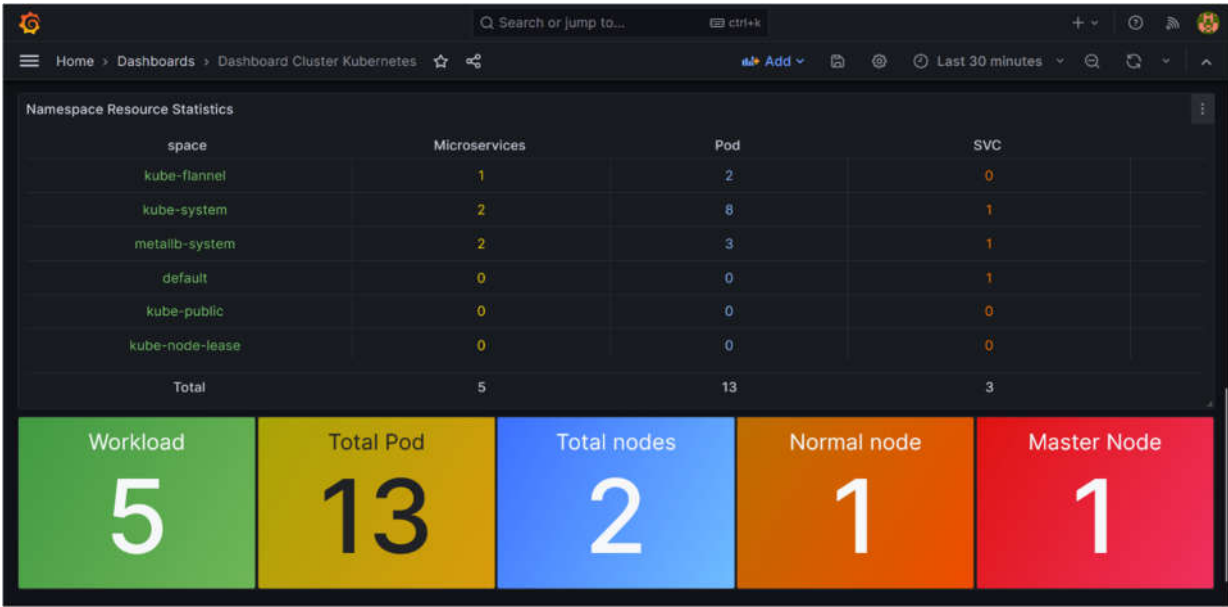
Grafana

1. Pengujian apakah Grafana dapat dijalankan di browser untuk dashboardnya



2. Pengujian pada Grafana apakah ia dapat membuat dan memvisualisasikan data sources Prometheus dalam bentuk dashboard





Tabel Hasil Pengujian

No	Aktivitas	Ekspektasi	Hasil
KUBERNETES			
1.	Uji koneksi antar node (Ping)	Dapat reply dari node yang dituju	Berhasil
2.	Uji koneksi dengan SSH	Dapat login ke node yang dituju	Berhasil
3.	Uji status Pod dan Node	Semua Pod/Node dalam status "Running"/"Ready"	Berhasil
4.	Uji eksposur layanan dengan LoadBalancer	Layanan dapat diakses dari luar	Berhasil
PROMETHEUS			
5.	Uji koneksi ke target	Prometheus dapat meng-scrape dari semua target	Berhasil
6.	Uji Prometheus Dashboard	Pengguna dapat mengakses dashboard	Berhasil
7.	Uji querying metrics	Prometheus dapat melakukan query data metrics	Berhasil
GRAFANA			
8.	Uji integrasi dengan Prometheus	Grafana dapat mengambil data dari Prometheus	Berhasil
9.	Uji pembuatan dashboard	Dashboard menampilkan data metrics	Berhasil
10.	Uji akses ke Grafana dashboard	Pengguna dapat mengakses dashboard	Berhasil

Referensi

<https://grafana.com/>

<https://prometheus.io/>

<https://github.com/kubernetes-sigs/metrics-server>

<https://kubernetes.io/>

<https://github.com/prometheus-operator/kube-prometheus/tree/main/examples>

<https://medium.com/@joudwawad/comprehensive-beginners-guide-to-kube-prometheus-in-kubernetes-monitoring-alerts-integration-4ade4fa8fa8c>

<https://github.com/grafana/grafana>

<https://docs.byteplus.com/id/docs/vmp/Common-metrics-of-kube-state-metrics>

<https://adinusa.id/course/courses/kubernetes-cluster-administration>

<https://adinusa.id/course/courses/monitoring-with-prometheus>